esade

Python for Data Science
Module 5

Data wrangling: grain, split-apply-combine, joins

Wednesday 11 November 2026

Alberto Cámara

Before we start: pull, then copy

From today **the demo is in the repository before class**. Follow along in your own copy.

```
GitHub Desktop → Fetch origin → Pull

sessions/m05/demo.ipynb  →  work/m05/demo.ipynb
sessions/m05/lab.ipynb   →  work/m05/lab.ipynb
```

- The *Your turn* cells are blank: fill them in with me
- After class the full version replaces it. Your copy in `work/` is never touched
- The **M04 lab solution** is now in `sessions/m04/lab-solution.ipynb`

The CFO wants one number

Total revenue

16.0 M BRL

Total revenue

20.3 M BRL

Same data, same week, two analysts. **Which one goes to the CFO?**

Two kinds of revenue

Name	Definition	Grain
Paid revenue	`payment_value` summed per order	an order
Item revenue	`price + freight_value` summed over item lines	an item line

- Paid revenue: what customers paid
- Item revenue: what each item sold for, the only one we can split **by seller or category**
- We use these two names, with these definitions, in every notebook from now on

Today

1. **Raw → clean**: a typed layer, written by code
2. **Grain**: what is one row?
3. **Split-apply-combine**: `groupby` with named aggregations
4. **Joins**: `merge` without losing or duplicating rows
5. **CP-pandas**: what you hand in next Tuesday

Lab: homework, due before M06 (ungraded).
It builds the course project's **order mart**.

Raw → clean

The layers

```
data/
  raw/     what Olist gave us. Read-only. Never edited
  clean/   the same tables, typed. One file per table. Written by code
  mart/    one table per question, at the question's grain     ← the lab
```

- Every number traces back to `raw/` through code you can re-run
- If `clean/` is deleted, Run All brings it back

The layer is a promise: **nothing in `clean/` was typed by hand.**

Why parquet?

CSV

- Text: every type is lost
- Dates come back as `object`
- 17.7 MB for orders

Parquet

- Stores the types
- Dates come back as dates
- 10.4 MB for orders

`pyarrow` has been in your `pds` env since M01: it's the engine pandas uses to read and write parquet.

Grain

What is one row?

Table	One row is…	Key
orders	an order	`order_id`
customers	a customer **per order**	`customer_id`
items	an item line of an order	`(order_id, order_item_id)`
payments	a payment of an order	*your turn*
reviews	a review, **not** one per order	—

- 99,441 `customer_id`s, but 96,096 people (`customer_unique_id`)
- 547 orders have more than one review

Test the grain, don't assume it

```
assert orders["order_id"].is_unique
assert customers["customer_id"].is_unique
assert not items.duplicated(subset=["order_id", "order_item_id"]).any()
```

- `is_unique`: one column
- `duplicated(subset=[...])`: a key made of several columns
- Silence means correct. An `AssertionError` stops the notebook **before** a wrong number is computed

An `assert` turns "I think one row is an order" into a check that fails loudly.

Split-apply-combine

Split · apply · combine

```
payments.groupby("payment_type").agg(
    paid_revenue=("payment_value", "sum"),
    payments=("payment_value", "size"),
    mean_installments=("payment_installments", "mean"),
)
```

- **Split** the rows by `payment_type`
- **Apply** `sum`, `size`, `mean` to each group
- **Combine** the results into one table, one row per group
- Named aggregations: `new_name=(column, function)`

`size` vs `count`

```
orders.groupby("order_status").agg(
    orders=("order_id", "size"),
    with_delivery_date=("order_delivered_customer_date", "count"),
)
```

- `size`: rows in the group
- `count`: **non-missing** values in the column
- `nunique`: distinct values (orders per seller, not item lines)

Changing the grain on purpose

```
paid_by_order = (
    payments.groupby("order_id")
    .agg(paid_revenue=("payment_value", "sum"))
    .reset_index()
)
assert paid_by_order["order_id"].is_unique
```

103,886 payments → 99,440 orders. A `groupby` on a key **builds a table at a new grain**.

Joins

`merge` with a contract

```
orders.merge(
    customers[["customer_id", "customer_state"]],
    on="customer_id",
    how="left",
    validate="one_to_one",
    indicator=True,
)
```

- `how="left"`: keep every order, even without a match
- `validate=`: our grain assumption, written into the join. Wrong → `MergeError`
- `indicator=True`: a `_merge` column says where each row came from
- Then `assert len(result) == len(orders)`

Who disappears in an inner join?

	rows
orders	99,441
item lines	112,650
`orders.merge(items)` (inner)	112,650
`orders.merge(items, how="left")`	113,425

775 orders have no items, mostly unavailable or cancelled. The inner join drops them without a word.

The 20.3 M BRL

```
items.merge(payments, on="order_id")["payment_value"].sum()   # 20,308,134.71
```

- 112,650 items × 103,886 payments → **117,601 rows**
- An order with 2 items and 2 payments becomes 4 rows: every payment counted twice
- A **fan-out**: many-to-many on `order_id`

```
items.merge(payments, on="order_id", validate="many_to_one")
# MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
```

"Aggregate first", and still wrong

```
items.merge(paid_by_order, on="order_id", validate="many_to_one")["paid_revenue"].su
# 20,308,134.71
```

The join is valid now. The total is **still 20.3 M**: the order total sits on every item line.

Sum a number only at its own grain. Paid revenue is an order number: sum it over orders.

To split by seller, use **item revenue**, which lives on the item line.

So, the CFO

Number	What it is
16.0 M	paid revenue, summed over orders ✓
15.8 M	item revenue: ~1% less (vouchers, instalment interest) ✓
20.3 M	a payment counted once per item line ✘

Each ✓ comes with its proof:

```
assert len(orders_with_paid) == len(orders)
by_states = revenue_by_state["paid_revenue"].sum()
assert abs(by_states - payments["payment_value"].sum()) < 0.01
```

The checklist

Step	Check
Write the clean layer	read it back: types survived
State the grain	`is_unique`, `duplicated(subset=…)`
Change the grain	`groupby(key).agg(...)`, then assert the key is unique
Join	`validate=`, `indicator=True`, assert the row count
Sum	only at the number's own grain; assert totals match the source

CP-pandas

CP-pandas · due Tue 17 Nov, 23:59

Individual. One notebook on Moodle. **5 items × 1 point**: your mark out of 5 is the % it adds.

Graded, not a progress check: each item is marked exactly by its *Met if* line.

1. **Clean orders**: typed, in `data/clean/`
2. **Grain**: state it, assert it, prove the category join keeps the row count
3. **Item revenue by category**, delivered orders, nothing lost (`"unknown"`)
4. **Monthly summary** with named aggregations
5. **Reproducible**: fresh kernel, Run All, ends with the `answers = {...}` cell

The full text and the `answers` template are on Moodle.

Lab: the order mart (homework, before M06)

`sessions/m05/lab.ipynb` → copy to `work/m05/lab.ipynb`. About an hour.

- **Part 1**: write the clean layer: items, payments, customers, reviews
- **Part 2**: assert each grain; keep the latest review per order
- **Part 3**: items and payments → one row per order
- **Part 4**: join it all onto orders → `data/mart/orders_enriched.parquet`

M06 and M07 chart this mart. Stuck? The solution comes out at the start of M06, and running it writes the same mart.

Next week: charts

- Plotly express: columns → x, y, colour
- Which chart fits which question
- Grouping by two keys, and shares of a total
- **The chart is a table**: check its numbers

Before then: CP-pandas, and the lab.

esade

See you next Wednesday

Everything is in the repository. Pull before class.

Bring questions.